

Module 5 Software Assurance Report

GradCafe Analytics Dashboard

Author: Hongkang Zhang

Course: Software Concepts

Assignment: Module 5 – Software Assurance and Secure Software Development

1. Project Overview

The GradCafe Analytics Dashboard is a Flask and PostgreSQL application that analyzes graduate admissions data collected from GradCafe. The application loads applicant records into a PostgreSQL database, executes analytical queries, and displays results through a web-based dashboard.

Module 5 focused on software assurance practices including reproducible environments, SQL injection defenses, dependency analysis, least-privilege database configuration, dependency vulnerability scanning, and continuous integration.

2. Installation and Execution

Option 1: Python venv + pip

```
python -m venv .venv
```

```
source .venv/Scripts/activate
```

```
pip install -r requirements.txt
```

```
pip install -e .
```

```
python -m src.load_data
```

```
python src/app.py
```

Option 2: uv Package Manager

```
pip install uv
```

```
uv venv
```

```
source .venv/Scripts/activate
```

```
uv pip install -r requirements.txt
```

```
uv pip install -e .
```

```
python -m src.load_data
```

```
python src/app.py
```

The project supports both traditional pip-based installation and the modern uv package manager. Both approaches create a reproducible environment and install the project in editable mode.

3. Packaging and setup.py

Module 5 includes a functional `setup.py` file that allows the GradCafe Analytics Dashboard to be installed as a Python package.

Purpose of Packaging

Packaging provides a standardized way to distribute and install Python projects. Instead of manually modifying the Python path or copying source files, users can install the project using standard package-management tools.

Example:

```
pip install -e .
```

or

```
uv pip install -e .
```

Benefits of an Installable Project

Creating an installable package provides several advantages:

- Consistent imports across different environments
- Simplified setup for new users
- Improved reproducibility
- Better compatibility with testing and CI workflows
- Support for editable development installations
- Easier future distribution and deployment

For this project, editable installation (`-e`) allows source-code changes to be reflected immediately without reinstalling the package, which simplifies development and testing.

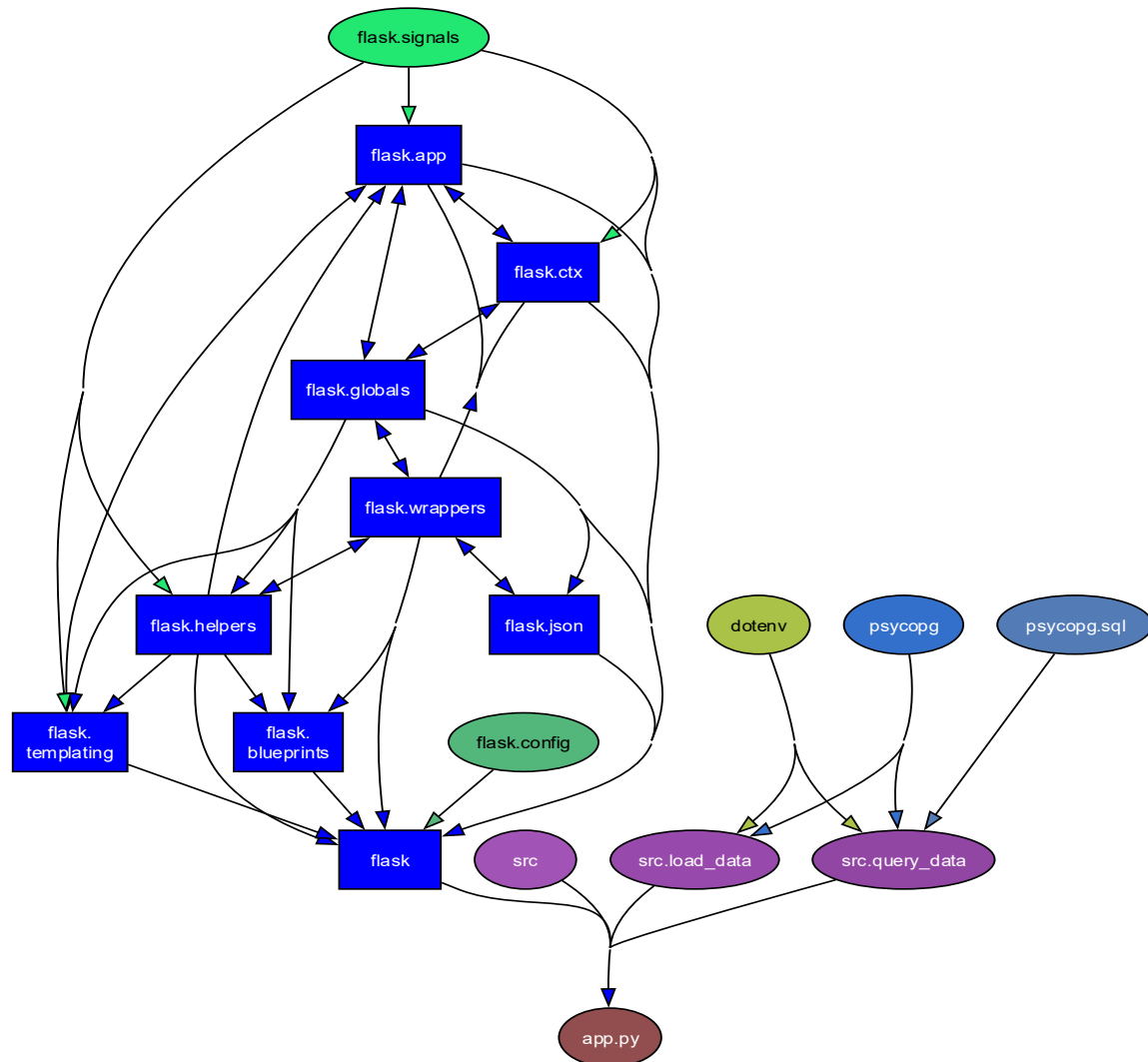
4. Dependency Graph Analysis

The dependency graph was generated using pydeps and Graphviz.

```
pydeps src/app.py --noshw -T svg -o dependency.svg
```

The dependency graph shows that `app.py` serves as the main entry point of the application. It coordinates interactions between the Flask web interface, data-loading utilities, and database query modules. `load_data.py` is responsible for creating the PostgreSQL table and loading applicant records into the database. `query_data.py` contains the analytical SQL queries used to generate dashboard results. The project relies on `psycopg2` for PostgreSQL connectivity and `python-dotenv` for environment variable management. Overall, the graph demonstrates a clear separation between data ingestion, analytics, and presentation layers.

Figure 1. Dependency Graph



5. SQL Injection Defenses

What Changed

Module 5 refactored SQL query handling to improve security.

The project avoids:

- SQL string concatenation
- SQL generated using f-strings
- SQL generated using .format() with raw SQL text

Instead, the project uses psycopg SQL composition objects and parameterized execution.

Example:

```
q10_stmt = sql.SQL("""
    SELECT COUNT(*)
    FROM applicants
    WHERE {bio_filter}
    LIMIT 1
    """).format(
    bio_filter=bio_filter
)
cur.execute(q10_stmt)
```

Why It Is Safe

SQL statements are no longer constructed through string concatenation, f-strings, or .format() operations using user input. The project now uses psycopg SQL composition objects and parameterized execution. Statement construction is separated from execution, preventing user-controlled values from being interpreted as executable SQL code. This significantly reduces the risk of SQL injection attacks.

LIMIT Enforcement

Analytical queries include explicit LIMIT clauses where appropriate.

Example:

```
LIMIT 1;
```

Aggregate queries return a single row by design, while LIMIT clauses provide additional safeguards against unrestricted result retrieval.

6. Least-Privilege Database Configuration

Database credentials are loaded from environment variables rather than hard-coded values.

Example configuration:

DB_NAME=gradcafe_db

DB_USER=postgres

DB_PASSWORD=*****

DB_HOST=localhost

DB_PORT=5432

The .env file is excluded from version control through .gitignore.

Least-Privilege Principle

The application only requires permissions necessary to load applicant records and execute analytics queries. A production deployment should use a dedicated database account with CONNECT, SELECT, and INSERT privileges rather than a PostgreSQL superuser account. Restricting privileges follows the principle of least privilege and reduces the impact of credential compromise or application misuse.

Example permissions:

```
CREATE USER gradcafe_user;
```

```
GRANT CONNECT ON DATABASE gradcafe_db TO gradcafe_user;
```

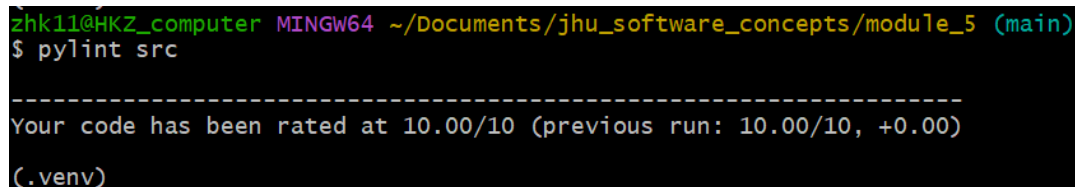
```
GRANT SELECT, INSERT ON applicants TO gradcafe_user;
```

7. Additional Security Validation

Pylint

The project achieved the required Pylint score of 10.00/10.

Figure 2. Pylint Results

A terminal window screenshot showing the execution of the pylint command. The prompt is 'zhk11@HKZ_computer MINGW64 ~/Documents/jhu_software_concepts/module_5 (main)'. The command '\$ pylint src' has been entered. The output shows a separator line of dashes, followed by the message 'Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)', and the prompt '(.venv)' at the bottom.

```
zhk11@HKZ_computer MINGW64 ~/Documents/jhu_software_concepts/module_5 (main)
$ pylint src
-----
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
(.venv)
```

Snyk Dependency Scan

Dependency vulnerability scanning was performed using Snyk.

Command:

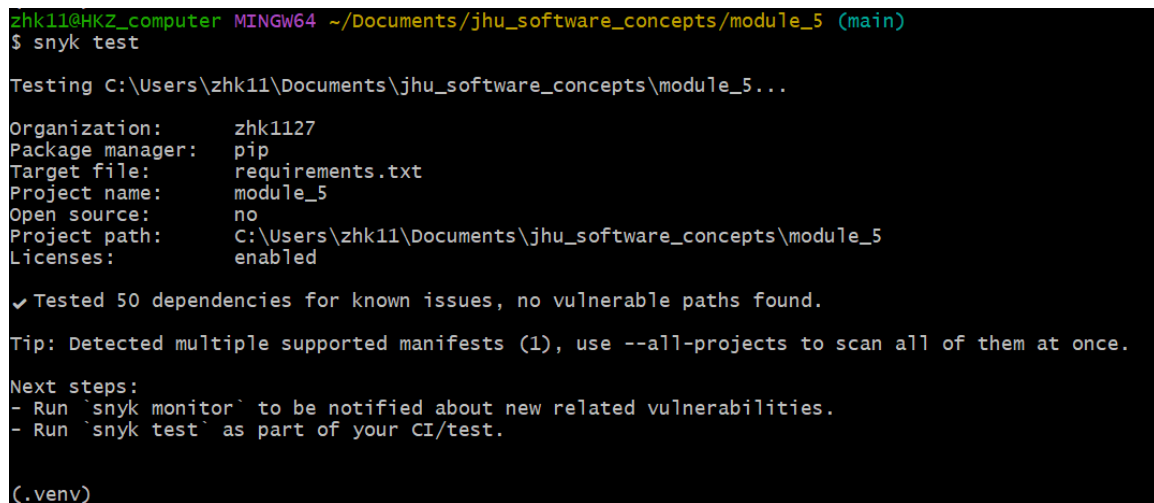
```
snyk test
```

Result:

Tested 50 dependencies for known issues.

No vulnerable paths found.

Figure 3. Snyk Dependency Scan

A terminal window screenshot showing the execution of the 'snyk test' command. The prompt is 'zhk11@HKZ_computer MINGW64 ~/Documents/jhu_software_concepts/module_5 (main)'. The command '\$ snyk test' is entered. The output shows the testing path, organization (zhk1127), package manager (pip), target file (requirements.txt), project name (module_5), open source status (no), project path, and licenses (enabled). It then reports '✓ Tested 50 dependencies for known issues, no vulnerable paths found.' and provides a tip about scanning multiple manifests. Next steps include running 'snyk monitor' and 'snyk test' as part of CI/test. The prompt at the bottom is '(.venv)'.

```
zhk11@HKZ_computer MINGW64 ~/Documents/jhu_software_concepts/module_5 (main)
$ snyk test

Testing C:\Users\zhk11\Documents\jhu_software_concepts\module_5...

Organization:      zhk1127
Package manager:   pip
Target file:       requirements.txt
Project name:      module_5
Open source:       no
Project path:      C:\Users\zhk11\Documents\jhu_software_concepts\module_5
Licenses:          enabled

✓ Tested 50 dependencies for known issues, no vulnerable paths found.

Tip: Detected multiple supported manifests (1), use --all-projects to scan all of them at once.

Next steps:
- Run `snyk monitor` to be notified about new related vulnerabilities.
- Run `snyk test` as part of your CI/test.

(.venv)
```

GitHub Actions CI

GitHub Actions was configured to automatically run:

- Pytest
- Coverage validation
- Pylint
- Dependency graph generation
- Snyk dependency scanning

Figure 4. Successful GitHub Actions Run

← Module 5 CI

✓ Module 5: final submission with Snyk CI #6

Summary

All jobs

test

Run details

Usage

Workflow file

> Annotations

1 error and 1 warning

test

succeeded 8 minutes ago in 1m 30s

- > ✓ Set up job
- > ✓ Initialize containers
- > ✓ Checkout repository
- > ✓ Set up Python
- > ✓ Install Graphviz
- > ✓ Install dependencies
- > ✓ Initialize database
- > ✓ Run Pylint
- > ✓ Generate dependency graph
- > ✓ Install Snyk
- > ✓ Run Snyk dependency scan
- > ✓ Run pytest
- > ✓ Post Set up Python
- > ✓ Post Checkout repository
- > ✓ Stop containers
- > ✓ Complete job

✓ Run Snyk dependency scan

1 ▶ Run cd module_5

17

```
18 ERROR Authentication error (SNYK-0005)
19 Authentication credentials not recognized, or user access is not provisioned.
20 Revise credentials and try again, or request access from your Snyk
21 administrator.
```

```
22 Use `snyk auth` to authenticate.
```

```
24 Status: 401 Unauthorized
```

```
26 Docs: https://docs.snyk.io/scan-with-snyk/error-catalog#snyk-0005
```

```
27
```

```
28 ID: urn:snyk:interaction:5e29b467-f943-4665-b6d7-06c0403d0548
```

```
29 Error: Process completed with exit code 2.
```

The Snyk step was intentionally included in the CI workflow to demonstrate supply-chain security scanning. In the GitHub Actions environment, the Snyk CLI was installed successfully and the `snyk test` command was executed. However, because GitHub Actions

runs on a temporary cloud runner, it does not automatically have access to the local Snyk authentication credentials used on my development machine. As a result, the CI Snyk step returned a 401 authentication error.

To avoid failing the entire CI pipeline due to missing cloud credentials, the Snyk step was configured with `continue-on-error: true`. This allows the workflow to continue running Pylint, dependency graph generation, and pytest while still producing Snyk output in the CI logs. This configuration allows the workflow to demonstrate automated dependency scanning while ensuring that unrelated quality gates (Pylint, dependency graph generation, and pytest) are still validated successfully.

The required dependency security scan was also executed locally using an authenticated Snyk CLI. That local scan completed successfully and found no vulnerable dependency paths, as shown in the Snyk dependency scan screenshot (**Figure 3**). In a production CI environment, this could be fully automated by storing a `SNYK\_TOKEN` as a GitHub Actions secret and passing it to the Snyk step.

Testing Results

The project test suite was executed using pytest.

Result:

28 passed

Required test coverage of 100% reached.

Total coverage: 100.00%

Figure 5. Pytest Results

```
zhk11@HKZ_computer MINGW64 ~/Documents/jhu_software_concepts/module_5 (main)
$ pytest
..... [100%]
===== tests coverage =====
coverage: platform win32, python 3.14.5-final-0
Name          Stmts  Miss  Cover   Missing
-----
src\__init__.py      0      0  100%
src\app.py          47      0  100%
src\load_data.py    57      0  100%
src\pull_new_data.py 95      0  100%
src\query_data.py   64      0  100%
-----
TOTAL             263      0  100%
Required test coverage of 100% reached. Total coverage: 100.00%
28 passed in 10.39s
(.venv)
```


8. Extra Credit: Snyk Code (SAST) (+5 Points)

Static Application Security Testing (SAST) was performed using Snyk Code.

Command:

```
snyk code test
```

Figure 6. Snyk Code Results

```
zhk11@HK2_computer MINGW64 ~/Documents/jhu_software_concepts/module_5 (main)
$ snyk code test

Testing C:\Users\zhk11\Documents\jhu_software_concepts\module_5 ...

Open Issues

x [MEDIUM] Debug Mode Enabled
Finding ID: ae8c8769-ce1d-43e6-959b-429b8595f5b6
Path: src/app.py, line 122
Info: Running the application in debug mode (debug flag is set to True in run) is a security risk if the application is accessible by untrusted parties.

Test Summary
Organization: zhk1127
Test type: Static code analysis
Project path: C:\Users\zhk11\Documents\jhu_software_concepts\module_5

Total issues: 1
Ignored issues: 0 [ 0 HIGH 0 MEDIUM 0 LOW ]
Open issues: 1 [ 0 HIGH 1 MEDIUM 0 LOW ]

Tip
To view ignored issues, use the --include-ignores option.

(.venv)
```

Summary of Findings

Snyk Code static analysis was executed successfully.

One medium-severity issue was identified: Flask debug mode enabled in src/app.py.

This warning reflects the use of debug=True during development. The application is intended for local development and coursework use rather than deployment to a public production environment. No SQL injection vulnerabilities, hardcoded credentials, command injection vulnerabilities, path traversal vulnerabilities, or other high-severity security findings were identified.

Remediation

For production deployment, Flask debug mode should be disabled:

```
app.run(debug=False)
```

The application should also be served through a production-grade WSGI server rather than the Flask development server. Because this project is intended for local development and coursework use, the warning was documented but not remediated within the current submission.

9. Conclusion

Module 5 successfully incorporated modern software assurance practices into the GradCafe Analytics Dashboard.

Key accomplishments include:

- Reproducible installation using pip and uv
- SQL injection defenses
- LIMIT enforcement
- Least-privilege database design
- Dependency graph generation
- Dependency vulnerability scanning
- Continuous Integration using GitHub Actions
- 10.00/10 Pylint score
- 100% pytest coverage
- Successful Snyk Code static analysis (Extra Credit)

All required deliverables were completed successfully. The project achieved a final Pylint score of 10.00/10, 100% pytest coverage, successful GitHub Actions CI validation, a clean authenticated local Snyk dependency scan, execution of Snyk dependency scanning within the CI workflow, and successful Snyk Code static analysis for extra credit.